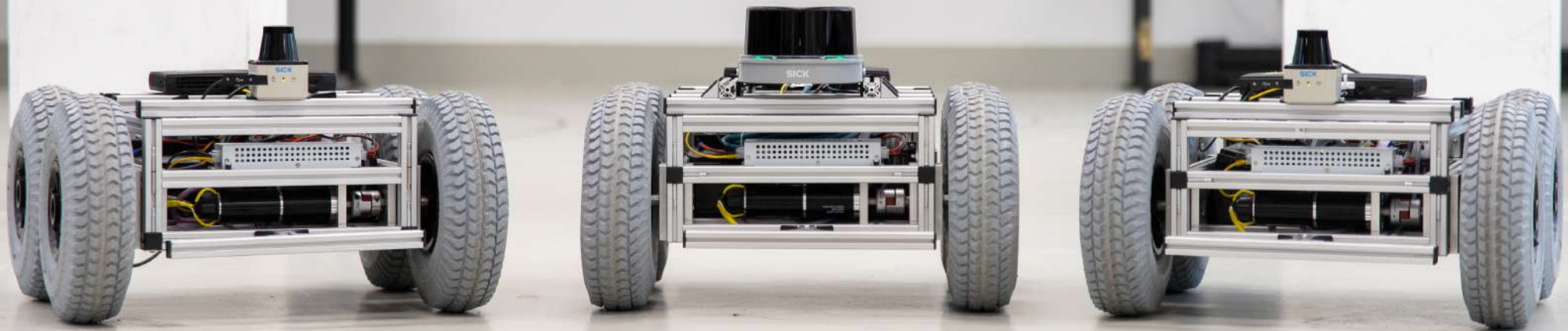


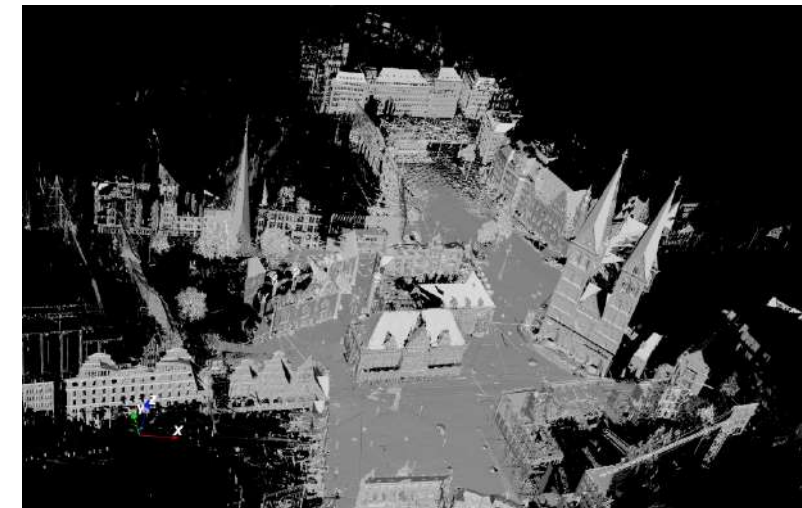
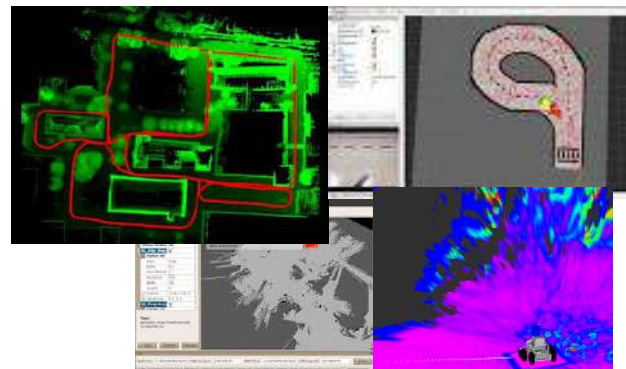
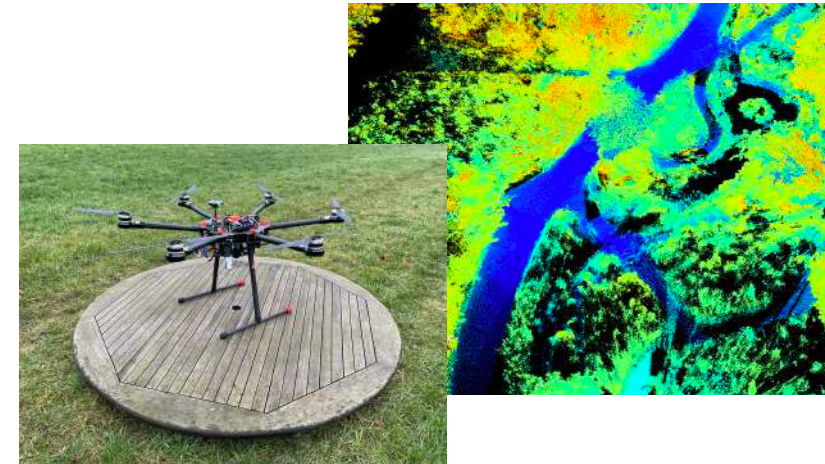
Robotik

Prof. Dr. Thomas Wiemann - FB AI - Robotik



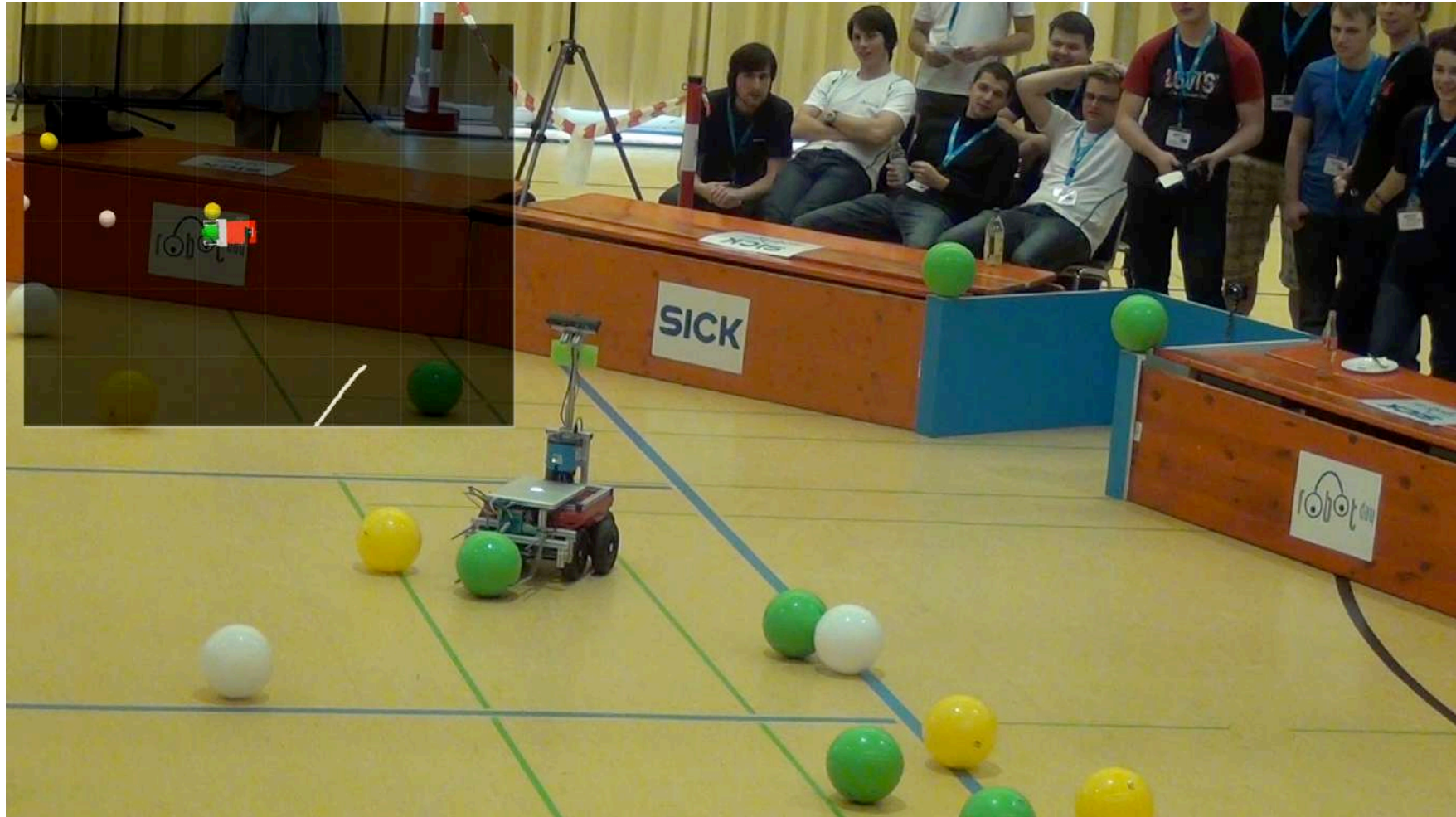
Hochschule Fulda
University of Applied Sciences







Worum geht es?



<https://youtu.be/MBWrW9AmFmU>

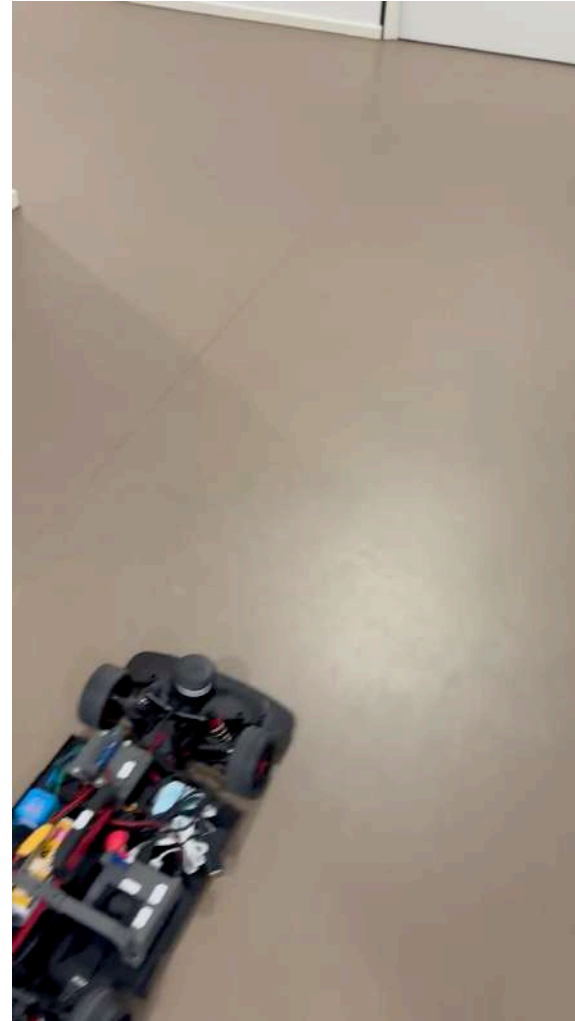


Roboter an der Hochschule





Roboter an der Hochschule





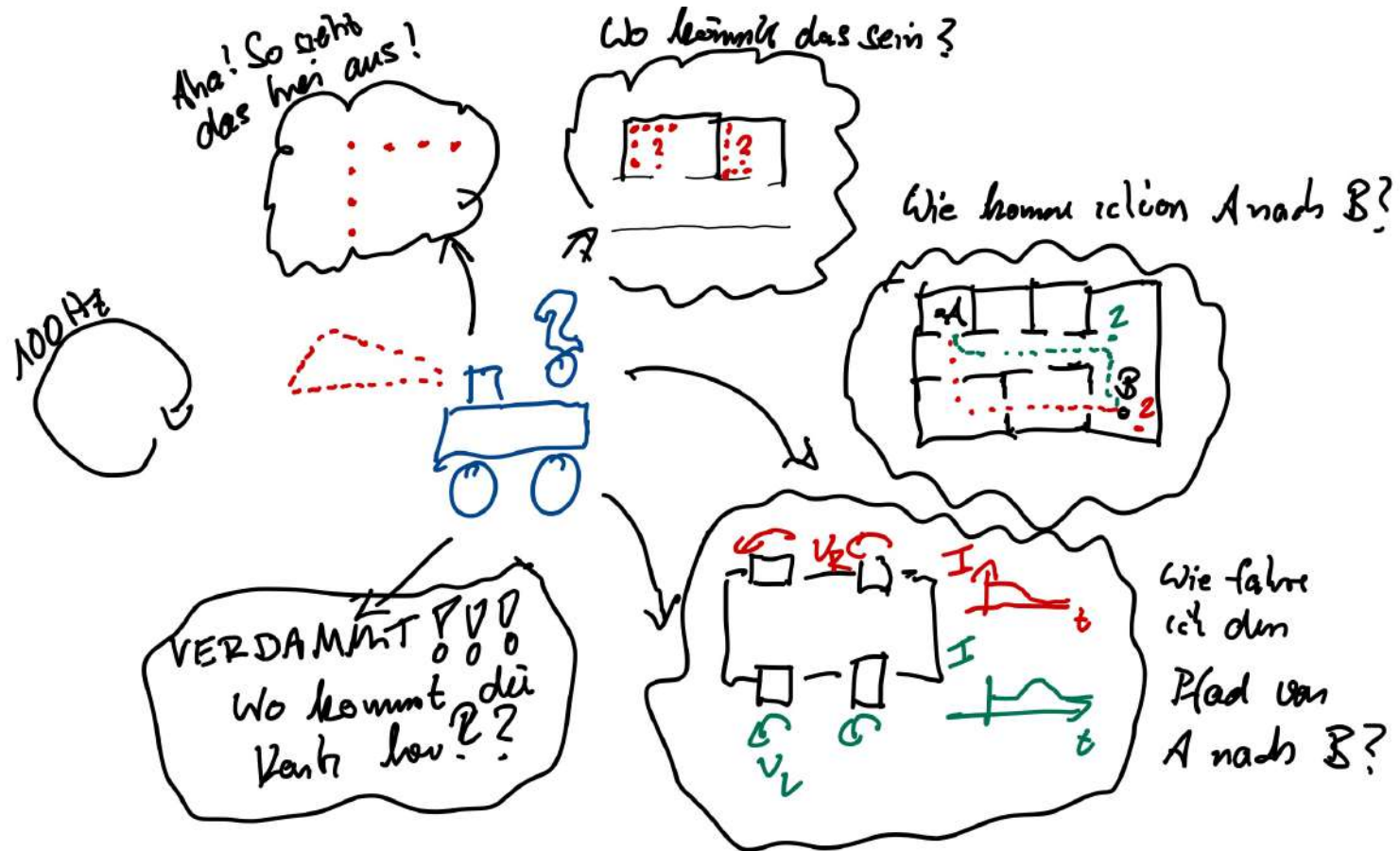
Aufbau eines mobilen Roboters, der in der Lage ist autonom kollisionsfrei von A nach B zu fahren

Themen / Inhalte

- ▶ Einarbeitung in das Robot Operating System ROS2
- ▶ Beschreibung und Simulation von Robotern
- ▶ Vorstellung erforderlicher Sensoren
- ▶ Filtern und Aufbereiten der Sensordaten
- ▶ Erarbeitung der Kinematik für einen Differenzialantrieb
- ▶ Lokalisierung in der Karte (inkrementell und global)
- ▶ Erstellung einer Umgebungskarte
- ▶ Implementierung eines einfachen Pfadplanungsalgorithmus' in der Karte



Ziele / Inhalte der Veranstaltung

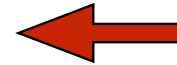




Komponente 1: Erlernen der Grundlagen

Es gibt 6 Übungsblätter, die immer aus zwei Teilen bestehen. Aufgaben werden in 4er-Gruppen bearbeitet. Jede Gruppe stellt zweimal im Semester einen Teil vor. Diese Präsentation wird bewertet. Wer drankommt, wird in der Übung entschieden (freiwillige Meldung oder Los). Es gibt einen opt-out Token, den man im Semester **vor** der Besprechung eines Teiles ziehen kann.

40 %



Teilt Euch innerhalb der Gruppen sinnvoll auf

Komponente 2: Theorie

Drei schriftliche Tests im Laufe des Semesters

30 %

Komponente 3: Praktische Anwendung

Miniprojekt (zwei Wochen): Anwendung des Roboters in praxisnahe Beispiel

30 %



Zeitlicher Ablauf

Woche	SU	Thema	ÜB	Thema	Portfolio
1	16. Apr	Intro / ROS2			
2	23. Apr	Sensoren: Inkrementalgeber, GPS, Laserscanner, ...	23.04 - 30.04	Einweisung Roboter / Bearbeitung Blatt 1	
3	30. Apr	Kinematiken / Diffetenzialantrieb	30.04 - 04.05	Präsentationen Blatt 1	Abgabe
4	07. Mai	Datenverarbeitung / Filterung	07.05 - 11.05	Bearbeitung Blatt 2 (Maifeiertag)	
5	14. Mai	Christi Himmelfahrt	14.05 - 18.05	Bearbeitung Blatt 2	
6	21. Mai	Wahrscheinlichkeiten, Bayes-Filter	21.05 - 25.05	Präsentationen Blatt 2 / Test 2	Abgabe / Test 1
7	28. Mai	Kalmanfilter 1	28.05 - 01.06	Präsentationen Blatt 3	Abgabe
8	04. Jun	Fronleichnam	04.06 - 08.06	Bearbeitung Blatt 4	
9	11. Jun	Scanmatching / Kalmanfilter 2	11.06 - 15.06	Präsentationen Blatt 4	Abgabe
10	18. Jun	Markow-Lokalisierung	18.06 - 22.06	Präsentationen Blatt 5	Abgabe
11	25. Jun	Monte-Carlo-Lokalisierung	25.06 - 29.06	Bearbeitung Blatt 6 / Test 2	Test
12	02. Jul	Planung / Navigation	02.07 - 06.07	Präsentationen Blatt 6	Abgabe
13	09. Jul	Vorstellung Projektthemen	09.07 - 13.07	Projektarbeit / Test 3	Test
14	16. Jul	Projektarbeit		Projektpräsentationen	



Prüfungsanmeldung



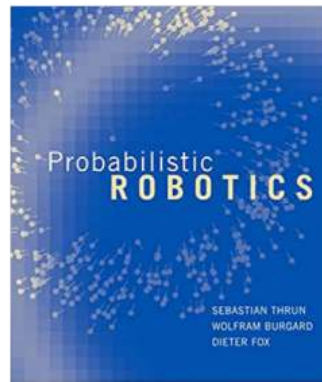
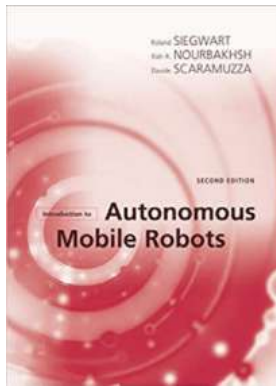


- ▶ Rechner im Linux-Lab und RobLab sind auf Inhalte vorbereitet, Labor ist offen
- ▶ Übungen an den Robotern im RobLab (Zugangsberechtigung erforderlich),
 - Arbeiten im Labor grundsätzlich immer möglich, aber ohne garantierte Betreuung
 - Es kann ein Transponder pro Gruppe beantragt werden, dazu Name und Matrikel per Mail an mich, ich bereite dann den Antrag vor
- ▶ Netzwerkzugang per Laptop zu den Robotern:
 - E-Mail mit Mac-Adresse an Michael Mecik
 - Wir tragen den dann morgen die Laptops ein
 - Passwort Netzwerk HSFD-M2M, Passwort: `IoTwithMachine2MachineOnly!`
 - Dann sollten Laptop / Roboter sich sehen können

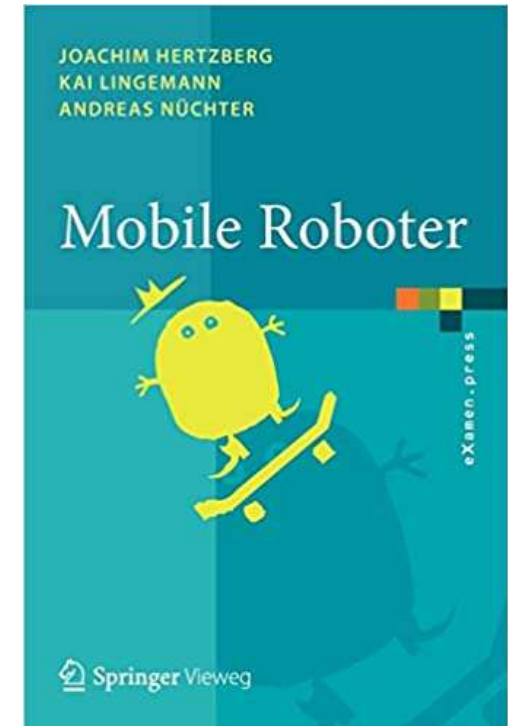


► Bücher:

- Die Folien enthalten die wesentlichen Inhalte der Vorlesung, kein Skript
- Inhalte basieren auf dem Robotik-Buch von Hertzberg, Lingemann, Nüchter
- Weitere Buchempfehlungen



Diese Bücher sind für's Selbststudium zur Ergänzung / Vertiefung des Vorlesungsstoffes gedacht. Es wird eventuell Leseaufgaben als Vorbereitung für die Übungen geben!





Gliederung

- 1. Einleitung**
2. Robot Operating System
3. Sensorik
4. Fortbewegung
5. Lokalisierung
6. Mapping
7. Navigation
8. Ausblick



- Typische Bausteine von Software zur Steuerung autonomer Roboter

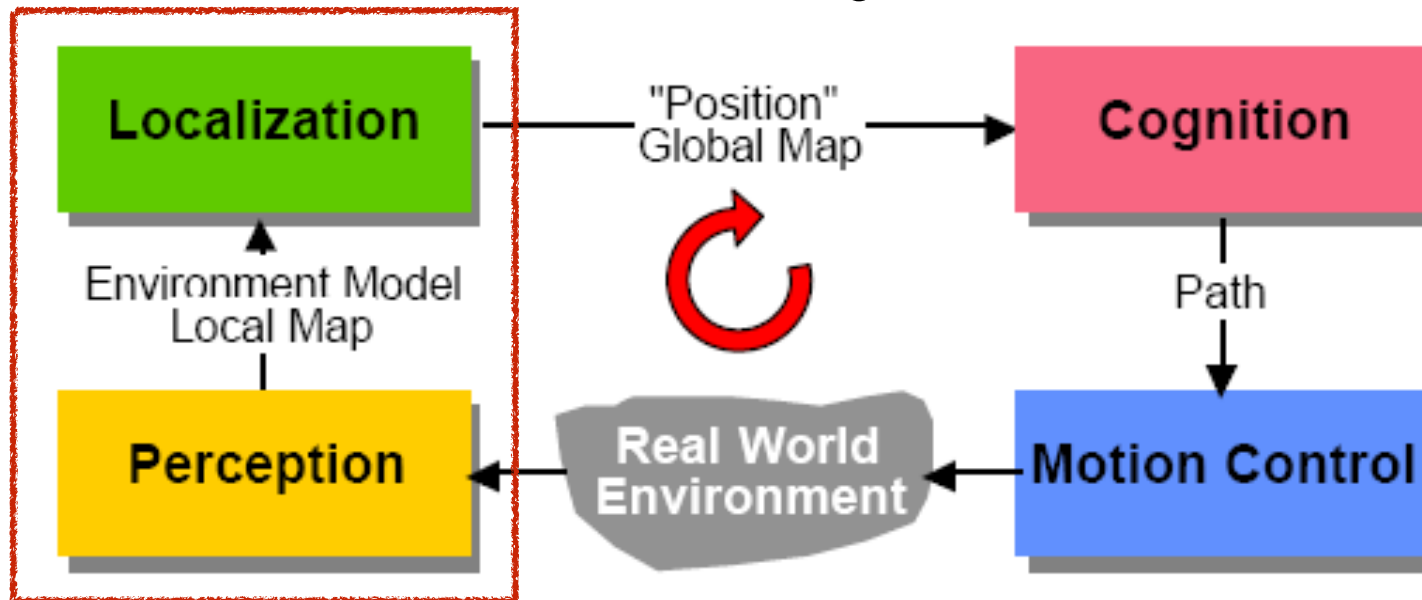


- In dieser Veranstaltung werden wir die aktuellen "Standardalgorithmen" zur Steuerung autonomer erarbeiten
- Als Framework verwenden wir ROS
- Einführung in Teil der nächsten Vorlesung, praktischer Einsatz in der Übung

ROS.org



- Typische Bausteine von Software zur Steuerung autonomer Roboter



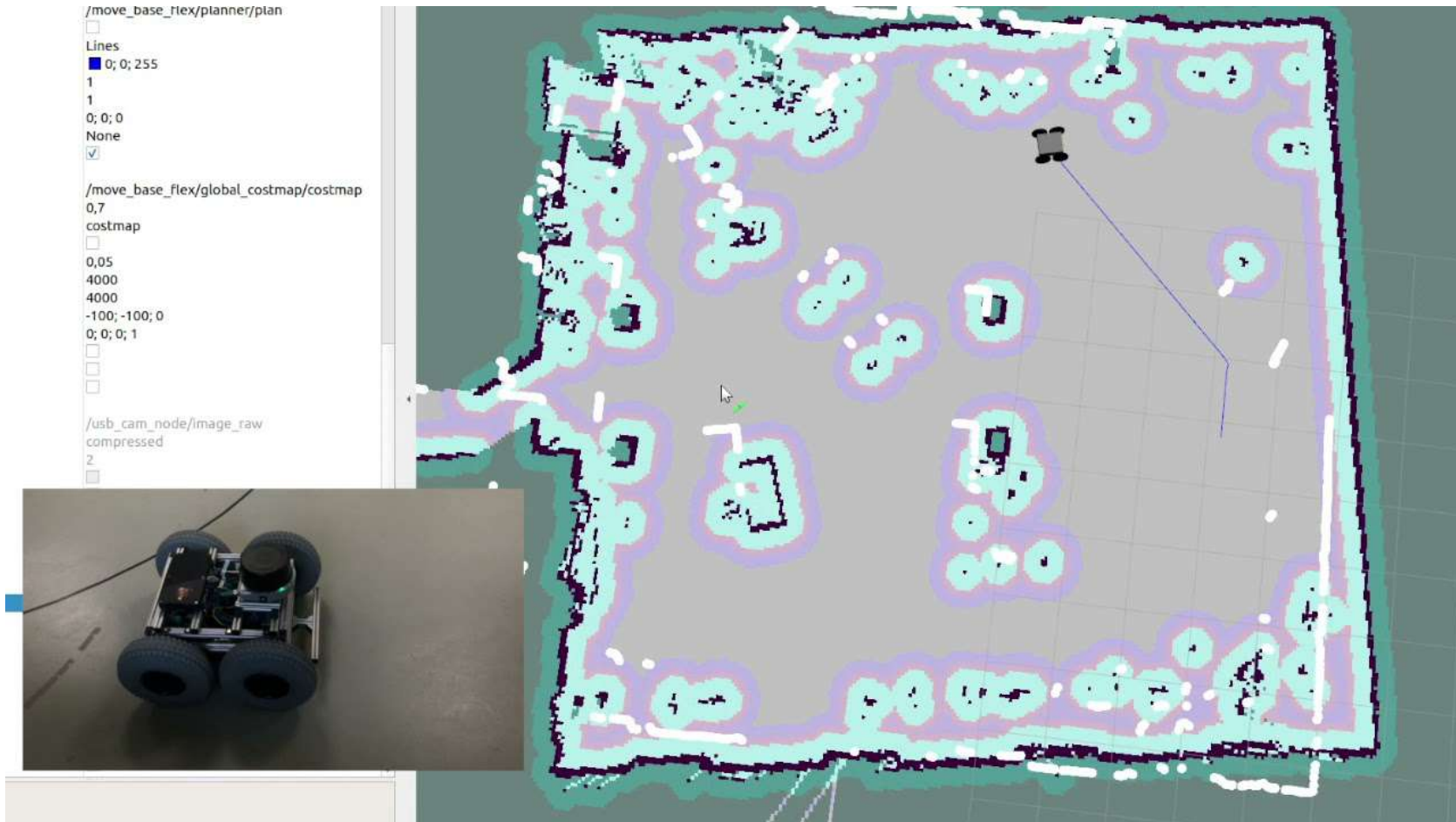
- In dieser Vorlesung werden wir die aktuellen “Standardalgorithmen” zur Robotersteuerung erarbeiten
- Als Framework verwenden wir ROS
- Einführung in Teil der nächsten Vorlesung, praktischer Einsatz in der Übung

ROS.org



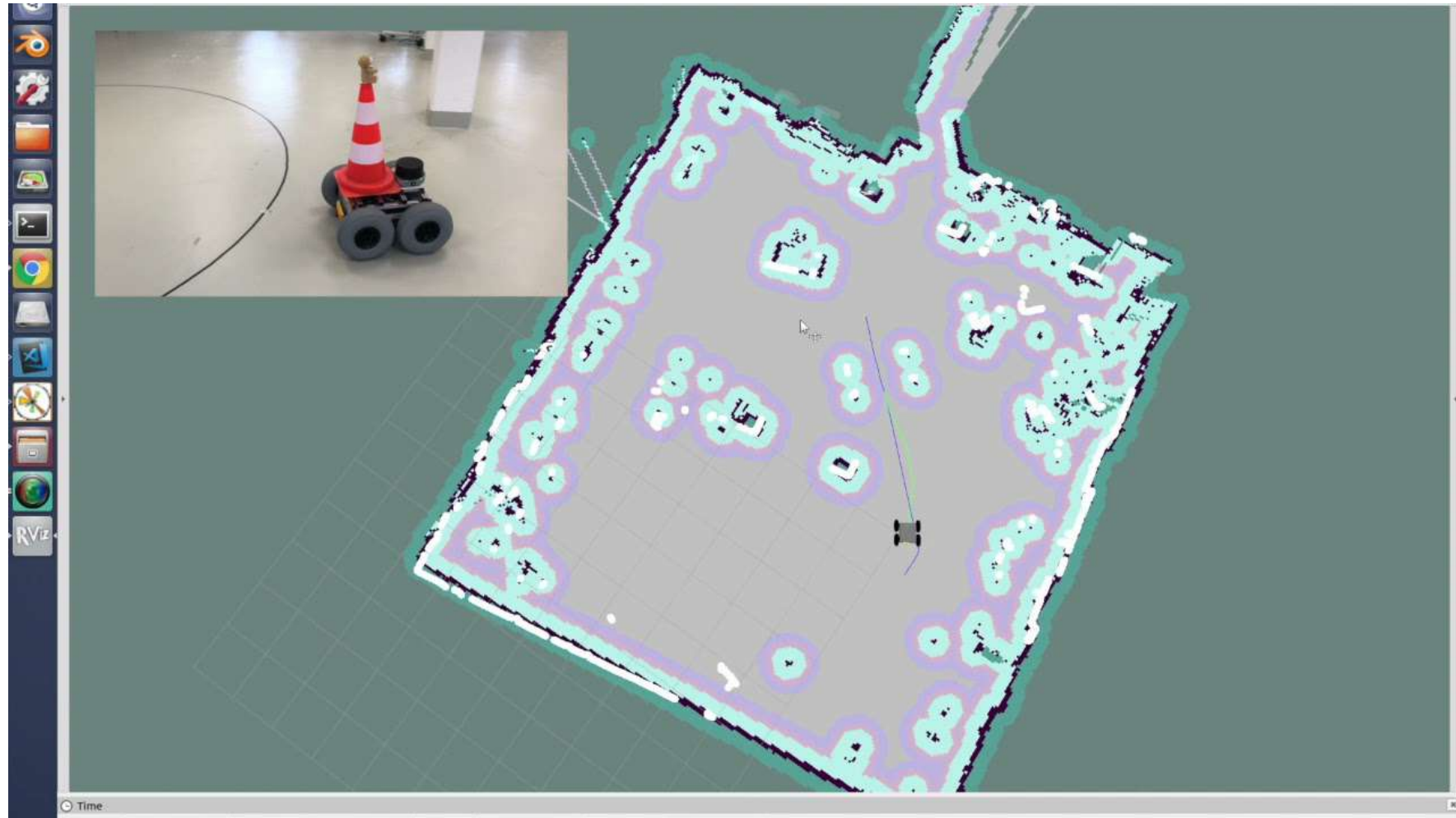
- ▶ Machen wir hier nicht, ist aber wichtig
- ▶ In unseren Robotern ist die Motorregelung eine Blackbox
- ▶ Bei Interesse: Regelungstechnik in ET

2D Robotik - Stand der Technik





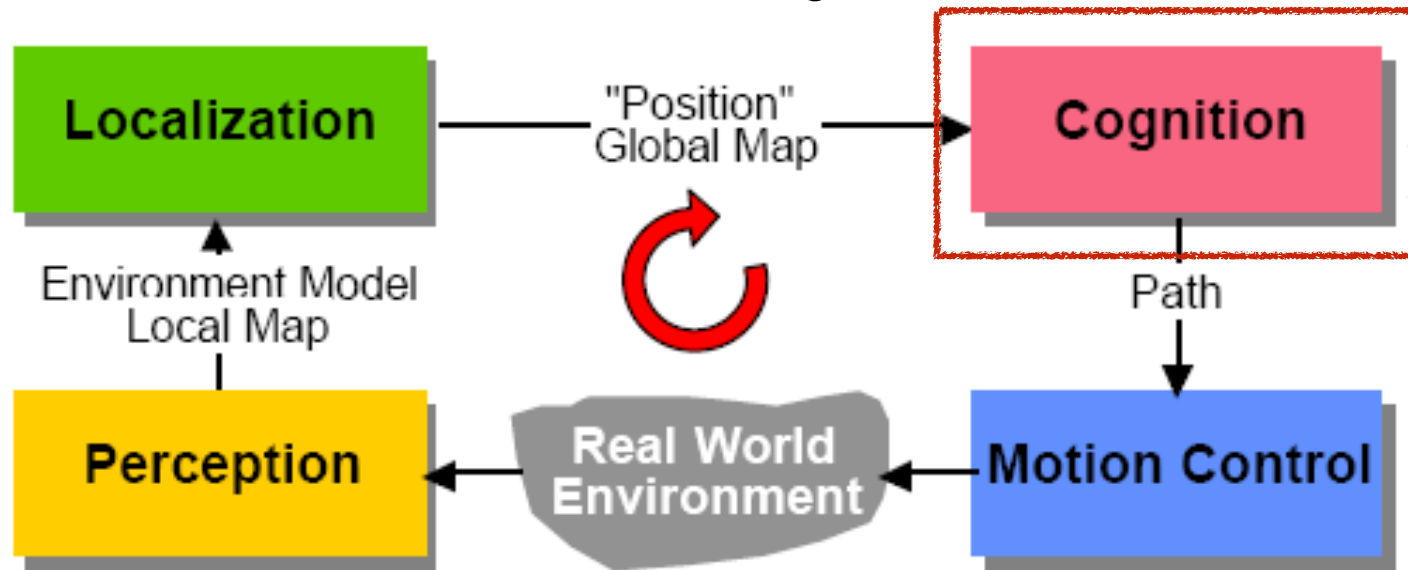
2D Robotik - Grenzen



<https://youtu.be/OCXRVx7zvgE>



- Typische Bausteine von Software zur Steuerung autonomer Roboter



- In dieser Vorlesung werden wir die aktuellen "Standardalgorithmen" zur Robotersteuerung erarbeiten
- Als Framework verwenden wir ROS
- Einführung in Teil der nächsten Vorlesung, praktischer Einsatz in der Übung

ROS.org



Beispiel Pfadplanung





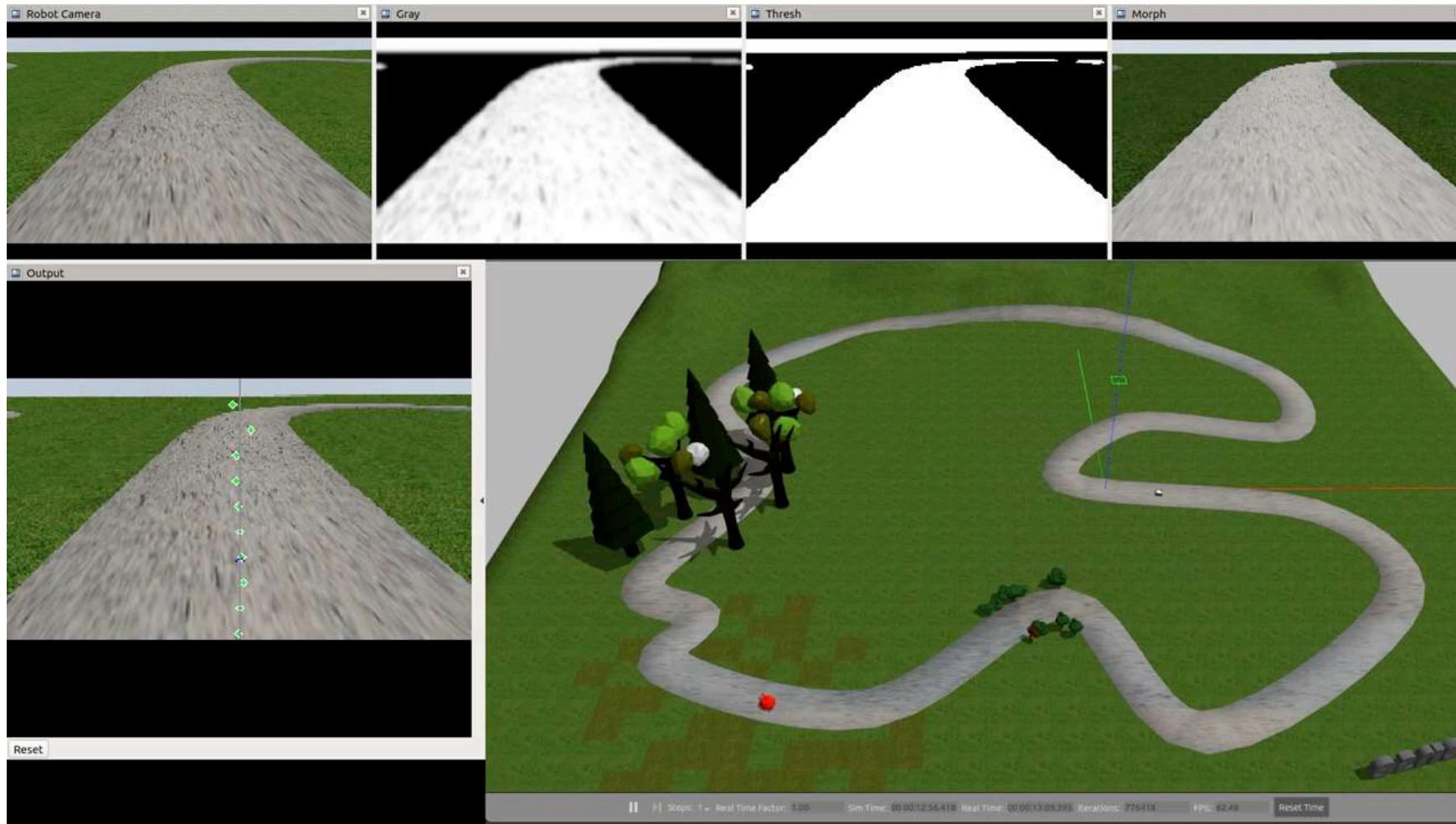
Simulationsumgebungen

- ▶ Software zur Steuerung von mobilen Robotern kann in Simulationsumgebungen entwickelt und erprobt werden
- ▶ Erlauben:
 - ▶ Sicheres Testen von neuen Algorithmen
 - ▶ Reproduzierbarkeit von Ergebnissen
 - ▶ Evaluation anhand so genannter "Ground Truth"
- ▶ Erfordert genaue Modellierung der Umgebung und der verwendeten Komponenten
 - ▶ Realistisches physikalisches Umgebungsmodell
 - ▶ Modellierung der Sensoren ihrer Funktionsweise
 - ▶ Präzises Robotermodell
- ▶ Physikalisch "korrekte" Berechnungen sind sehr rechenaufwendig
- ▶ Simulationen können Experimente an realen Robotern nicht ersetzen





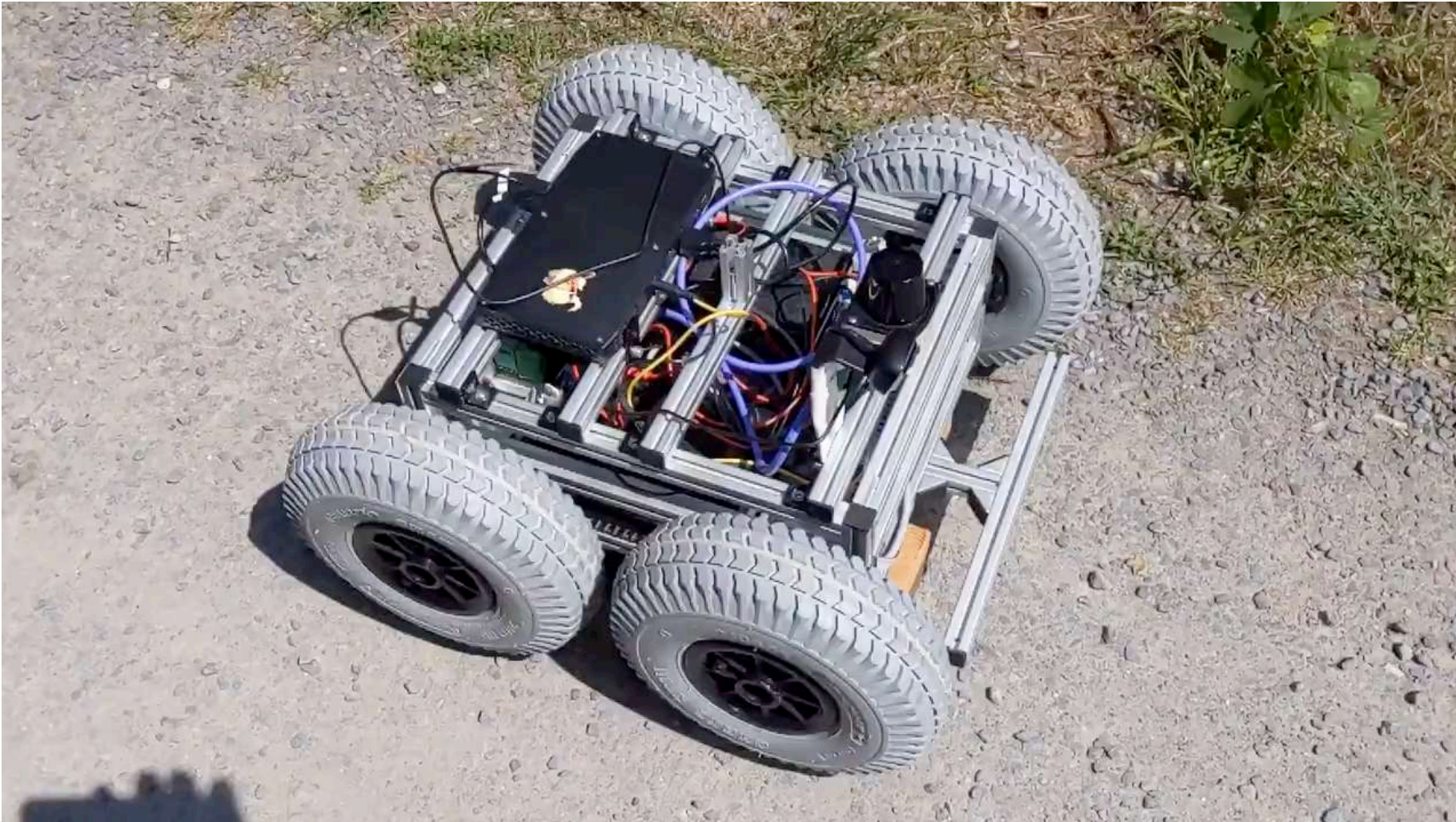
Simulationsumgebungen - Beispiel



<https://youtu.be/r8PvBtes8T8>



Testen auf dem realen Roboter



<https://youtu.be/Vc9BV66Wm6s>



- ▶ Einzelne Robotik-Methoden und -Techniken sind bereits heute weit verbreitet.
- ▶ Den Nutzern fallen die eingesetzten Robotik-Methoden und -Techniken meist gar nicht auf !
- ▶ Die Robotik bekommt Einfluss, wenn sie zu Massenprodukten oder -leistungen führt, die ohne ihre Techniken nicht denkbar waren
- ▶ Diese Produkte müssen nicht nach „Roboter“ aussehen!
- ▶ In dieser Vorlesung werden hauptsächlich die algorithmischen Grundlagen zur Steuerung autonomer mobiler Roboter behandelt
- ▶ ... in 2D...
- ▶ ... 3D ist Gegenstand aktueller Forschung



Gliederung

1. Einleitung
- 2. Robot Operating System**
3. Sensorik
4. Fortbewegung
5. Lokalisierung
6. Mapping
7. Navigation
8. Ausblick



Gliederung

2. Robot Operating System

1. Grundlagen

2. Nachrichten

3. Transformationen

4. Robotermodellierung

6. Mapping

7. Navigation

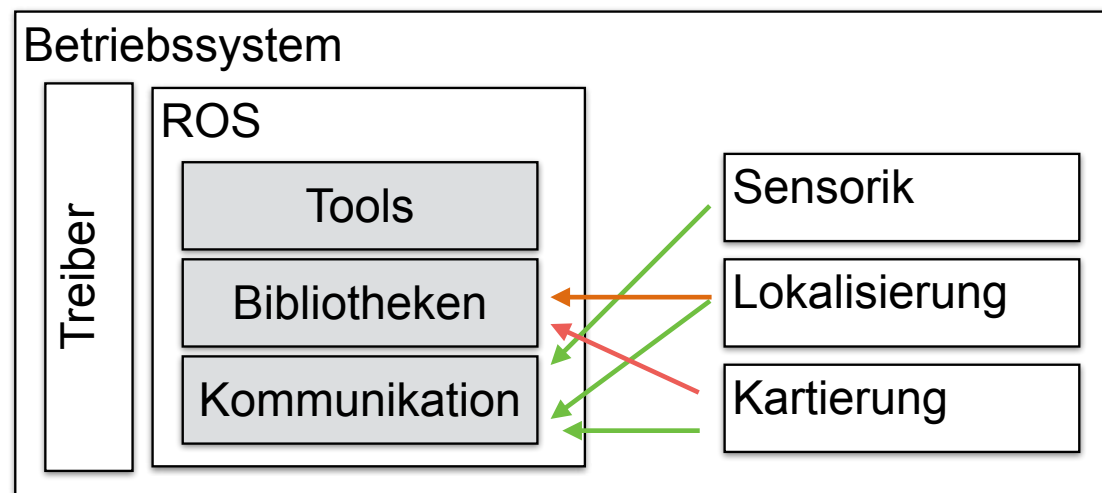
8. Ausblick





Was ist ROS

- ▶ Open-Source Robot Operating System
- ▶ Bibliotheken und Hilfsprogramme, die es erleichtern Software für verschiedene Plattformen zu entwickeln
- ▶ Ursprünglich 2007 beim Stanford AI-Lab entwickelt
- ▶ Ab 2009 Hauptentwicklung bei Willow Garage
- ▶ Seit 2013 in Händen der Open Source Robotics Foundation (OSRF)



- Algorithmik
- Infrastruktur

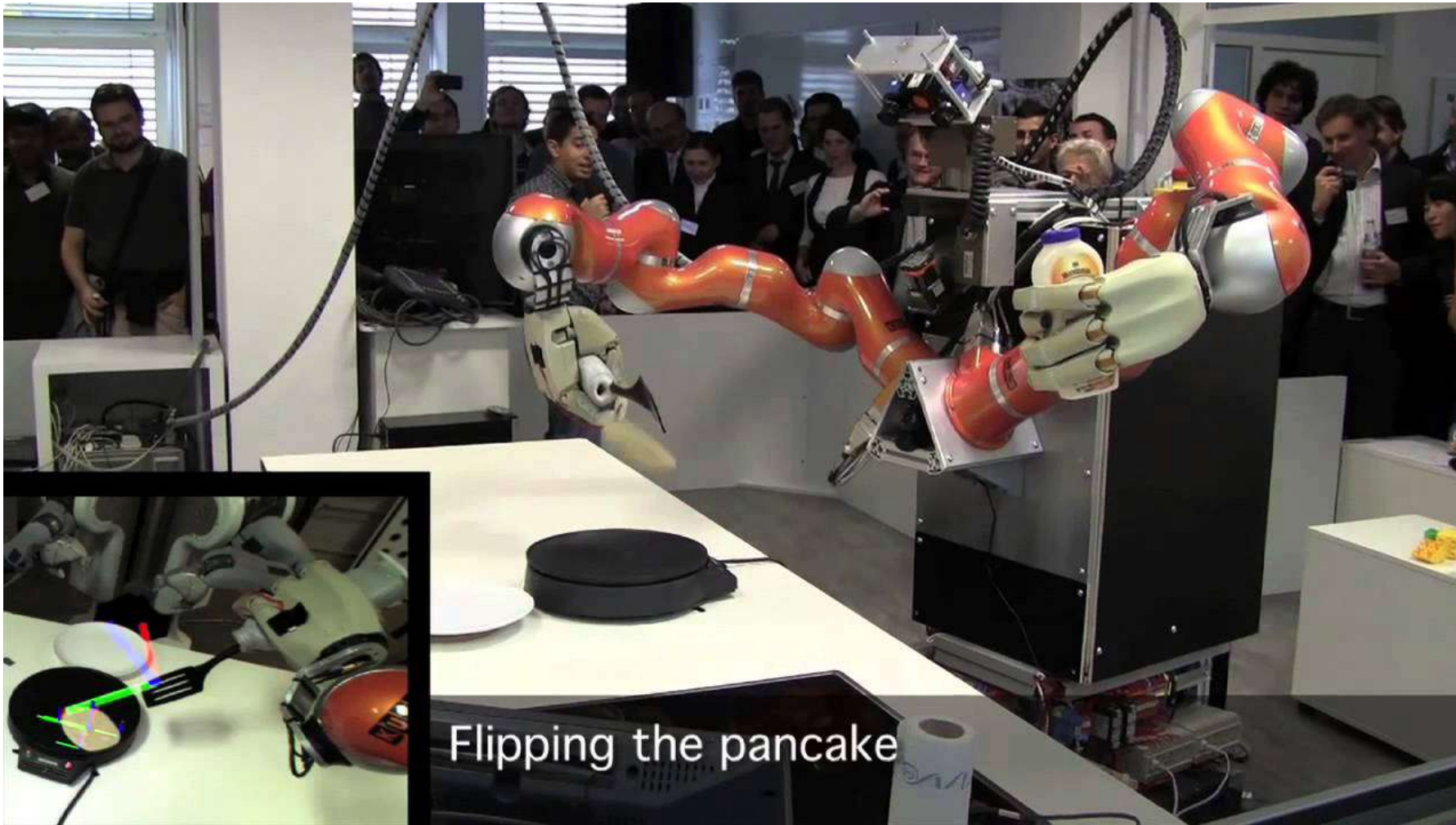


- ▶ Keine Standards / Konventionen für Koordinatensysteme, Repräsentation von Sensordaten und Austauschformaten
- ▶ Wenig wiederverwertbarer Code
- ▶ Viele Grundlegende Funktionalitäten mussten selbst entwickelt werden:
 - Treiber für Komponenten
 - Interfaces für den Datenaustausch zwischen Komponenten
 - Betriebssystemspezifische Anpassungen
 - Interprozesskommunikation
 - Ressourcenhandling
 - ...
- ▶ Wenn diese Implementiert wurden, mussten die Standardalgorithmen für das jeweilige System neu programmiert werden
- ▶ Wenn ein neuer Roboter ins Labor kam ➡ Goto #1



Stop Reinventing the Wheel





<https://youtu.be/PGaXiLZD2KQ>



ROS - Distributionen

Distro	Release date	Poster	Tuturtle, turtle in tutorial	EOL date
ROS Noetic Ninjemys (Recommended)	May 23rd, 2020			May, 2025 (Focal EOL)
ROS Melodic Morenia	May 23rd, 2019			May, 2023 (Bionic EOL)
ROS Lunar Loggerhead	May 23rd, 2017			May, 2019
ROS Kinetic Kame	May 23rd, 2016			April, 2021 (Xenial EOL)
ROS Jade Turtle	May 23rd, 2015			May, 2017
ROS Indigo Igloo	July 22nd, 2014			April, 2019 (Trusty EOL)
ROS Hydro Medusa	September 4th, 2013			May, 2015

Distro	Release date	Logo	EOL date
Humble Hawksbill	May 23rd, 2022		May 2027
Galactic Geckelone	May 23rd, 2021		November 2022
Foxy Fitzroy	June 5th, 2020		May 2023
Eloquent Elusor	November 22nd, 2019		November 2020
Dashing Diademata	May 31st, 2019		May 2021
Crystal Clemmys	December 14th, 2018		December 2019



Die zwei Gesichter von ROS

- ▶ Das “Betriebssystem”...
 - Stellt Komponenten zur bereit, die bei direkter Implementierung eigentlich vom Betriebssystem übernommen würden
 - Beispiele:
 - Hardware-Abstraktion
 - Low-level Treiber
 - Nachrichtenversand zwischen Prozessen
 - Paketmanagement
 - Gemeinsam benötigte Funktionalität
- ▶ Die Software-Seite...
 - stellt Bibliotheken zur Verfügung
 - stellt Pakete mit Implementierungen von Nutzern zur Verfügung
 - z.B. Vision, SLAM, Planung usw.
- ▶ Im Zusammenspiel ermöglicht dies dank der gemeinsamen Schnittstellen Software platformübergreifend wiederzuverwenden



► **Peer-to-peer**

- Ein ROS-System besteht aus vielen kleinen Programmen “Nodes”
- Die Knoten verbinden sich miteinander und tauschen kontinuierlich Nachrichten aus
- Kommunikation via TCP/IP
- Die Knoten müssen nicht zwangsläufig auf demselben Rechner laufen

► **Werkzeugbasiert**

- ROS stellt viele nützliche Tools zur Verfügung, die helfen, den Zustand des Systems zu inspizieren
- Visualisierung, Logging, Anzeigen von Datenströmen

► **Bindings für verschiedene Sprachen**

- ROS-Komponenten können in jeder Sprache entwickelt werden, für die es entsprechende Client-Libraries gibt
- Hauptsächlich C++ und Python aber auch Java, MATLAB, LISP und andere

► **Kapselung**

- Softwarepakete sollen eigene Bibliotheken sein, die dann in ROS gekapselt werden

► **Frei und Open-Source, Community-Repos**



Der ROS Computation Graph

- Die Steuerung eines Roboters erfolgt durch den ROS Computation Graph: Die Gesamtheit von miteinander kommunizierenden Microservices (Nodes)
- Die Nodes kommunizieren nachrichtenbasiert.
- Die Verbindung (Welche Node kommuniziert mit welcher anderen) wird über topics gesteuert.
- Eine Node kann unterschiedlichste Aufgaben haben (Treiber, Lokalisierung, Planung,...)

